

第一章 基本概念

Ajou University

目录

概述：系统生命周期

指针与动态内存分配

算法描述

数据抽象

性能分析

性能测量

概述：系统生命周期

- ❖ 将大型计算机程序视为一个由许多复杂交互部分组成的系统
- ❖ 系统在开发过程中会经历一个称为**系统生命周期**的过程



概述：系统生命周期

1) 需求 (Requirements)

- 一组用于定义项目目标的规格说明
- 描述系统的输入与输出

2) 分析 (Analysis)

- 将问题分解为可管理的子问题；可采用**自底向上**或**自顶向下**方法
- 自底向上 (bottom-up) :
传统且较朴素的方法，早期侧重于编码细节
- 自顶向下 (top-down) :
从最终产品的目标出发，将问题划分为若干模块，
生成用于系统设计的结构图或功能图

概述：系统生命周期

3) 设计(Design)

- 考虑数据对象及其上的操作
- 创建抽象数据类型 (ADT)，给出算法的规格说明
- 与具体编程语言无关；推迟实现层面的决策

4) 细化与编码(Refinement and Coding)

- 选择数据对象的具体表示方式
- 为每个操作编写具体算法

概述：系统生命周期

5) 验证(Verification)

- **正确性证明(Correctness Proofs):通过数学方法证明程序的正确性**
 - 对于大型项目而言，进行形式化证明较为困难
 - 选择已被证明正确的算法可以减少错误
- **测试(Testing):需要可运行的程序代码以及一组测试数据**
 - 测试数据应尽可能覆盖所有情况
 - 除验证正确性外，还应测试程序的运行时间（性能）
- **错误排除 / 调试(Error Removal / Debugging):修正已发现的错误**
 - 与“意大利面式代码”（结构混乱、耦合度高的代码）相比，结构清晰、良好文档化并划分为相互独立模块、通过参数交互的程序更易于调试
 - 如果每个模块能够单独测试后再进行系统集成，调试工作会更加高效

指针与动态内存分配

❖ 指针(Pointer)

- 指针类型变量的实际值是一个内存地址
- 两个基本运算符
 - &: 取地址运算符
 - *: 解引用运算符 (或: 间接访问运算符)
- 特殊值
 - NULL(0): 表示不指向任何对象或函数

❖ 动态内存分配

- malloc: 根据需求分配一定数量的内存空间
 - 例) `int *pi = (int*)malloc( sizeof(int) );`
- free: 将已分配的内存空间归还给系统
 - 例) `free( pi );`

算法

❖ 定义

- 算法是 一组有限的指令序列，只要严格执行这些指令，就能完成某个特定任务

❖ 算法的五个基本特性(Requirements)

- Input (输入): 算法具有零个或多个外部提供的输入量
- Output (输出): 算法至少产生一个输出量
- Definiteness (确定性): 算法的每一步指令都必须明确、无歧义
- Finiteness (有穷性): 算法必须在执行有限步之后终止
- Effectiveness (有效性): 算法中的每条指令都必须足够基本、可行，能在有限时间内实际执行

示例：选择排序

- ❖ 设计算法，将包含 $n \geq 1$ 个整数的集合进行排序
- 一个简单的思路：在当前尚未排序的部分中找到**最小值**，将其放到已排序列表的下一个位置

i	[0]	[1]	[2]	[3]	[4]
-	30	10	50	40	20
0	10	30	50	40	20
1	10	20	50	40	30
2	10	20	30	40	50
3	10	20	30	40	50

示例：选择排序

❖ 算法

```
for (i=0;i<n;i++){  
    Examine list[i] to list[n-1] and suppose  
    that the smallest integer is at list[min];  
  
    Interchange list[i] and list[min];  
}
```

两个子任务

- 找到当前最小的整数
- 将它与 list[i] 交换位置

示例：选择排序

```
void sort(int list[], int n)
{
    int i, j, min, temp;
    for (i = 0; i < n-1; i++) {
        min = i;
        for (j = i+1; j < n; j++)
            if (list[j] < list[min])
                min = j;
        SWAP(list[i], list[min], temp);
    }
}
```

两个子任务

- 找到当前最小的整数
- 将它与 list[i] 交换位置

递归算法 Recursive Algorithm

- ❖ 递归(Recursion):递归是一种常见的简化方法, 通过将问题拆分成同类型的子问题来解决

E.g.: $n! = n \times (n-1)!$, $\sum_{i=1}^n i = n + \sum_{i=1}^{n-1} i$

- ❖ 函数会对规模更小的输入 (如 $n - 1$) 再次调用自身, 直到达到“基本情况(base case)”为止

Example示例: Power_of_2 (自然数 k)

Input: 一个自然数 k

Output: 2^k

Algorithm (recursive):

```
if k = 0, then return 1;  
else return 2*Power_of_2(k - 1)
```

```
int a = Power_of_2(4);    ←16  
      2*Power_of_2(3)    ←2*8  
      2*Power_of_2(2)    ←2*4  
      2*Power_of_2(1)    ←2*2  
      2*Power_of_2(0)    ←2*1
```

递归算法 Recursive Algorithm

❖ 优点与缺点

- 优点 (**Pros**) : 函数定义更自然、表达更直观
- 缺点 (**Cons**) : 相比迭代方式, 递归需要更多的内存(调用栈开销更大)

```
int factorial(int n) {  
    if(n == 0) { return 1; }  
    else {  
        return n * factorial(n-1);  
    }  
}
```

```
int fibonacci(int n) {  
    if(n == 0){ return 0;}  
    else if(n == 1) { return 1; }  
    else {  
        return (fibonacci(n-1) + fibonacci(n-2));  
    }  
}
```

数据抽象

❖ 数据类型

- 数据类型是 一组对象(objects) 和 对这些对象进行操作的操作集(operations) 的集合
- 例如, **int** 数据类型包含对象 {0, +1, -1, +2, -2, ..., INT_MAX, INT_MIN} 以及操作 +, -, *, /, and %

❖ C语言中的数据类型

- 基本数据类型: char, int, float and double
- 复合数据类型:
 - 数组 Arrays: 由同一基本类型的元素构成
 - 结构体 Struct: 由不同类型的元素组合而成

❖ 了解数据类型对象的底层表示

- 可能有用, 但也潜在危险
- 将数据类型对象的表示方式对用户隐藏, 只通过暴露的函数操作对象, 这是良好的设计策略

抽象数据类型 Abstract Data Type (ADT)

❖ 定义

- 一种数据类型，其特点是：对象及其操作的“规范”与对象的“表示方式”和操作的“实现方式”严格分离
- 例：Ada: package, C++ and Java: class

❖ 规范 vs 实现方式

- 操作规格说明
- 包括：函数名、参数类型、返回类型、函数的行为描述
- 不包括：内部数据结构表示方式或实现细节

示例：自然数抽象数据类型

ADT *NaturalNumber* is

objects: an ordered subrange of the integers starting at zero and ending at the maximum integer (*INT_MAX*) on the computer

functions:

for all $x, y \in \text{NaturalNumber}$, $TRUE, FALSE \in \text{Boolean}$

and where $+$, $-$, $<$, and $==$ are the usual integer operations.

<i>NaturalNumber</i>	Zero ()	::= 0
<i>Boolean</i>	Is_Zero(x)	::= if (x) return FALSE else return TRUE
<i>Boolean</i>	Equal(x, y)	::= if (x == y) return TRUE else return FALSE
<i>NaturalNumber</i>	Add(x, y)	::= if ((x+y) <= INT_MAX) return x+y else return INT_MAX
<i>NaturalNumber</i>	Successor(x)	::= if (x == INT_MAX) return x else return x+1
<i>NaturalNumber</i>	Subtract(x, y)	::= if (x < y) return 0 else return x-y

end *NaturalNumber*

“is defined as”

Transformers

性能分析

❖ 评判一个程序的标准

- 程序是否满足任务最初的需求说明?
- 程序是否正确运行?
- 程序是否包含说明文档, 说明如何使用以及其工作方式?
- 程序是否合理使用函数来构建清晰的逻辑结构?
- 程序代码是否易于阅读?
- 程序是否高效利用主存与辅存?
- 程序的运行时间是否能满足任务需求?

与培养
良好编程风格
相关

关注
性能评估

❖ 性能评估的两个不同领域

- 性能分析(与机器无关)
 - 空间复杂度与时间复杂度
- 性能测量(依赖机器环境)
 - 用于发现程序中效率低下的代码片段

空间复杂度 Space Complexity

❖ 定义

- 程序运行到结束所需要的内存量

$$S(P) = c + S_p(I)$$

- 固定空间需求 Fixed space requirements (c)
 - 与输入输出的数量和大小无关
 - 指令空间（程序代码占用空间）
 - 简单变量、固定大小结构体变量、常量所占空间
- 可变空间需求 Variable space requirements ($S_p(I)$)
 - Depend on the particular instance I of the problem being solved
 - 取决于所求解问题的具体实例 I
 - 输入输出的数量、大小、取值等
 - 例) 包含 n 个元素的数组 $\rightarrow n$ 就是该实例的特征
 - 递归函数空间
 - 递归栈空间、参数、局部变量、返回地址等

示例

❖ 简单算数函数

```
float abc (float a, float b, float c) {  
    return a+b+b*c+(a+b-c)/(a+b)+4.00;  
}
```

$$S_{abc}(I) = 0$$

❖ 迭代函数

```
float sum (float list[], int n) {  
    float tempsum=0; int i;  
    for (i=0;i<n;i++) { tempsum+= list[i]; }  
    return tempsum;  
}
```

$$S_{sum}(I) = 0$$

示例

所需空间

1. 参数
2. 局部变量
3. 返回地址

❖ 递归函数

```
float rsum (float list[], int n) {  
    if (n) return rsum(list, n-1) + list[n-1];  
    return 0;  
}
```

$$S_{\text{rsum}}(n) = 12n$$

Type	Name	Number of bytes
parameter : array pointer	list []	4
parameter : integer	n	4
return address : (used internally)		4
TOTAL per recursive call		12

时间复杂度

❖ 定义

- 程序从开始到结束所需要的 计算机时间

$$\begin{aligned}T(P) &= c + T_P(I) \\&= \text{编译时间} + \text{运行时间} \\&\approx \text{运行时间} \\&\approx \text{所执行的操作数量}\end{aligned}$$

- 程序P的总时间 $T(P)$, 由两部分组成: 它的编译时间 c 以及它的运行(执行)时间 $T_P(I)$
- 固定的时间需求
 - 编译时间 (c), 与输入实例的特性无关。
- 可变的时间需求
 - 运行(执行)时间 T_P

程序步骤

❖ 定义

- 一个在语法或语义上有意义的程序片段,
其执行时间不依赖于输入实例的特性

❖ 迭代求和一个数列

```
float sum(float list[ ], int n)
{
    float tempsum = 0;
    count++; /* for assignment */
    int i;
    for (i = 0; i < n; i++)
    {
        count++; /*for the for loop */
        tempsum += list[i]; count++; /* for assignment */
    }
    count++; /* last execution of for */
    count++; /* for return */
    return tempsum;
}
```

逐一统计每个操作步骤

$2n + 3$ steps

程序步骤

❖ Tabular Method :另一种获得步骤计数的方法

1. 确定每条语句所贡献的步骤总数

- 每次执行的步骤数 (s/e) x 频率Frequency

每条语句被执行的次数

2. 将所有语句的贡献相加

Program sum 的步骤计数表

Statement	s/e	Frequency	Total Steps
Float sum (float list[], int n)	0	0	0
{	0	0	0
float tempsum = 0;	1	1	1
int i;	0	0	0
for (i=0; i < n ; i ++)	1	n+1	n+1
tempsum += list[i];	1	n	n
return tempsum;	1	1	1
}	0	0	0
Total			2n + 3

渐进表示法 Asymptotic Notation

❖ 动机

- 用来判断一个算法在规模变大时是否实用
- 例) 比较 $c_1n^2 + c_2n$ 与 c_3n 的时间复杂度
 - 当 n 足够大时, c_3n 会比 $c_1n^2 + c_2n$ 更快
 - 当 n 较小时, 两者都有可能更快
 - $c_1=1, c_2=2, c_3=100 \rightarrow c_1n^2 + c_2n \leq c_3n \text{ for } n \leq 98$
 - $c_1=1, c_2=2, c_3=1000 \rightarrow c_1n^2 + c_2n \leq c_3n \text{ for } n \leq 998$
 - 平衡点
 - 无论 c_1, c_2 , and c_3 是多少, 总会存在一个 n 使得 c_3n 永远比 $c_1n^2 + c_2n$ 更快
- 引入三种步骤计数方法
 - 最好情况、最坏情况、平均情况

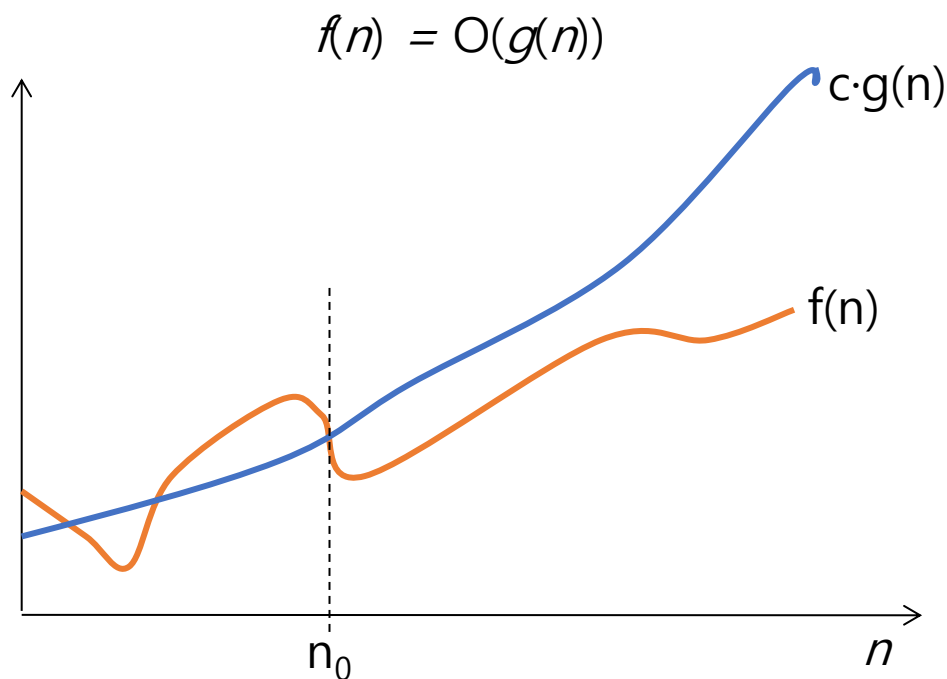
渐进表示法 Asymptotic Notation

- ❖ O (Big-Oh) $f(n) = O(g(n))$
“小于或等于” 的关系(上界)
- ❖ Ω (Big-Omega) $f(n) = \Omega(g(n))$
“大于或等于” 的关系(下界)
- ❖ Θ (Big-Theta) $f(n) = \Theta(g(n))$
“等于” 的关系(紧缺界/同阶)

大O表示法 Big-Oh

❖ 定义: $f(n) = O(g(n))$ 当且仅当存在正常数 c 和 n_0 使得 $f(n) \leq c \cdot g(n)$ 对所有 $n, n \geq n_0$ 都成立

If $f(n) = a_m n^m + \dots + a_1 n + a_0$, then $f(n) = O(n^m)$



Examples

$$3n+2=O(n)$$

$$: 3n+2 \leq 4n \text{ for } n \geq 2$$

$$3n+3=O(n)$$

$$: 3n+3 \leq 4n \text{ for } n \geq 3$$

$$100n+6=O(n)$$

$$: 100n+6 \leq 101n \text{ for } n \geq 10$$

$$10n^2+4n+2=O(n^2)$$

$$: 10n^2+4n+2 \leq 11n^2 \text{ for } n \geq 5$$

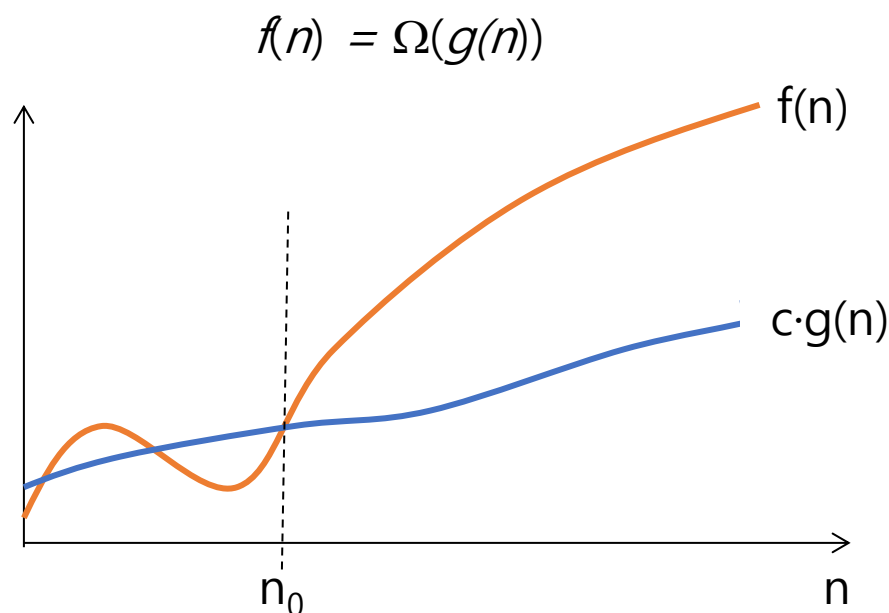
$$6 \cdot 2^n + n^2 = O(2^n)$$

$$: 6 \cdot 2^n + n^2 \leq 7 \cdot 2^n \text{ for } n \geq 4$$

大Ω Omega

❖ 定义: $f(n) = \Omega(g(n))$ 当且仅当存在正常数 c 和 n_0 使得 $f(n) \geq cg(n)$ 对所有 $n, n \geq n_0$ 都成立

If $f(n) = a_m n^m + \dots + a_1 n + a_0$ and $a_m > 0$, then $f(n) = \Omega(n^m)$



Examples

$$3n+2 = \Omega(n)$$

$$: 3n+2 \geq 3n \text{ for } n \geq 1$$

$$3n+3 = \Omega(n)$$

$$: 3n+3 \geq 3n \text{ for } n \geq 1$$

$$100n+6 = \Omega(n)$$

$$: 100n+6 \geq 100n \text{ for } n \geq 1$$

$$10n^2+4n+2 = \Omega(n^2)$$

$$: 10n^2+4n+2 \geq n^2 \text{ for } n \geq 1$$

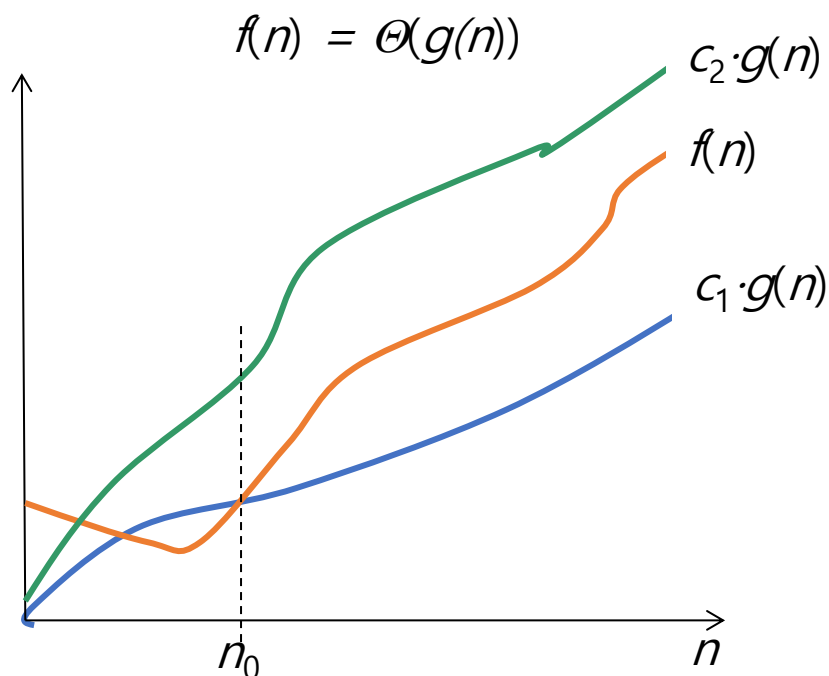
$$6 \cdot 2^n + n^2 = \Omega(2^n)$$

$$: 6 \cdot 2^n + n^2 \geq 2^n \text{ for } n \geq 1$$

大Θ Theta

❖ 定义: $f(n) = \Theta(g(n))$ 当且仅当存在正常数 c_1 , c_2 , 和 n_0 使得 $c_1g(n) \leq f(n) \leq c_2g(n)$ 对所有 $n, n \geq n_0$ 都成立

If $f(n) = a_m n^m + \dots + a_1 n + a_0$ and $a_m > 0$, then $f(n) = \Theta(n^m)$



Examples

$$3n+2 = \Theta(n)$$

$$10n^2+4n+2 = \Theta(n^2)$$

$$6 \cdot 2^n + n^2 = \Theta(2^n)$$

$$3n+2 = O(n^2), \text{ but } 3n+2 \neq \Theta(n^2)$$

实际复杂度 Practical Complexities

		Instance characteristic n					
Time	Name	1	2	4	8	16	32
1	Constant	1	1	1	1	1	1
$\log n$	Logarithmic	0	1	2	3	4	5
n	Linear	1	2	4	8	16	32
$n \log n$	Log linear	0	2	8	24	64	160
n^2	Quadratic	1	4	16	64	256	1024
n^3	Cubic	1	8	64	512	4096	32768
2^n	Exponential	2	4	16	256	65536	4294967296
$n!$	Factorial	1	2	24	40326	20922789888000	26313×10^{33}

Figure 1.7 Function values

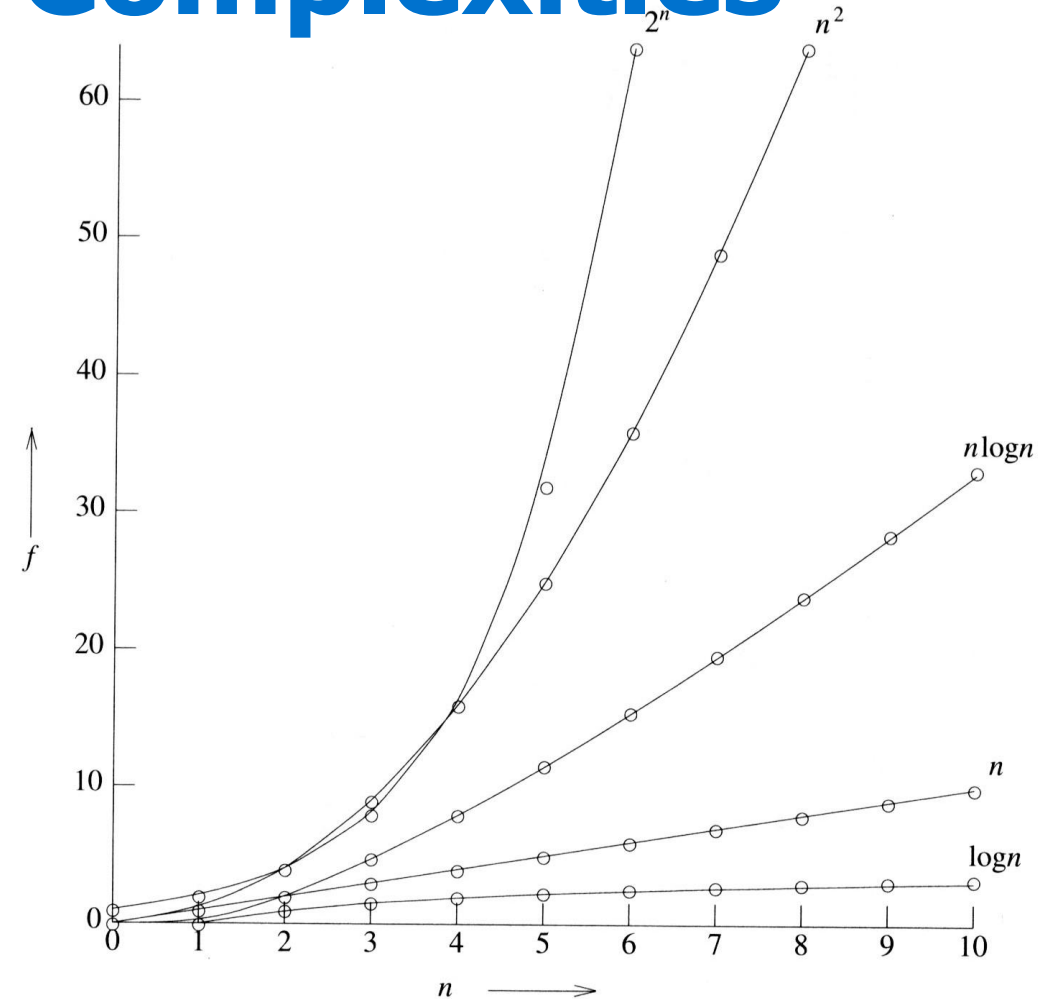


Figure 1.8 Plot of function values

性能测量 Performance Measurement

❖ Clocking (计时)

- 虽然性能分析为我们评估算法的空间和时间复杂度提供了一个强大的工具，但在某个时刻我们也必须考虑算法在我们的机器上是如何执行的
 - 这种考量使我们从分析领域转移到测量领域

Event timing in C

C语言标准库
`#include <time.h>`

	Method 1	Method 2
Start timing	<code>start = clock();</code>	<code>start = time(NULL);</code>
Stop timing	<code>stop = clock();</code>	<code>stop = time(NULL);</code>
Type returned	<code>clock_t</code>	<code>time_t</code>
Result in seconds	<code>duration = ((double) (stop-start)) / CLK_TCK;</code>	<code>duration = (double) difftime(stop,start);</code>